



Fullstack Web Development

Session 7 | Week 7/Day 1 | 120 min

React JS - Part 02

Instructor: Ms. Liev Nova



090667393



State Management in React

State management is the way an application **stores, reads, and updates data** that controls what users see in the UI.

In Simple Words:

State = data that changes in your app

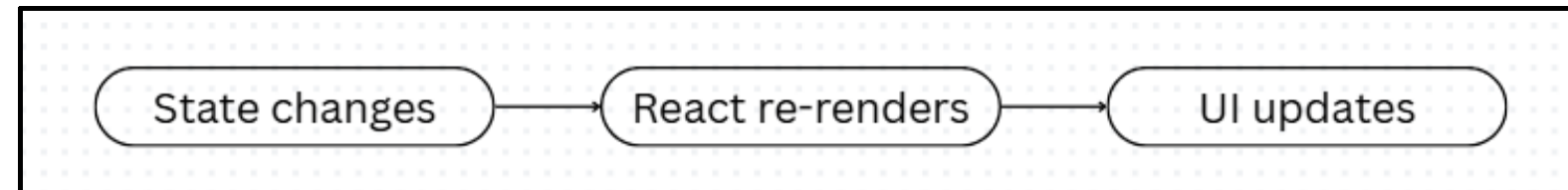
Example:

- user input
- button click result
- form data
- loading status

State Management in React

In ReactJS — **State Management** means how React applications **store, update, and share data between components**

How It Affects UI:



Example:

```
const [count, setCount] = useState(0);
```

- **count** = current data
- **setCount()** = update data

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **What is State?**

- State — Data Inside a Component
 - Stored inside a component
 - Can change over time
 - When state changes → component re-renders
 - Each component has its own state

- **useState Hook:**

```
const [value, setValue] = useState(initialValue);
```

- **value** → current state
- **setValue** → function to update state

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

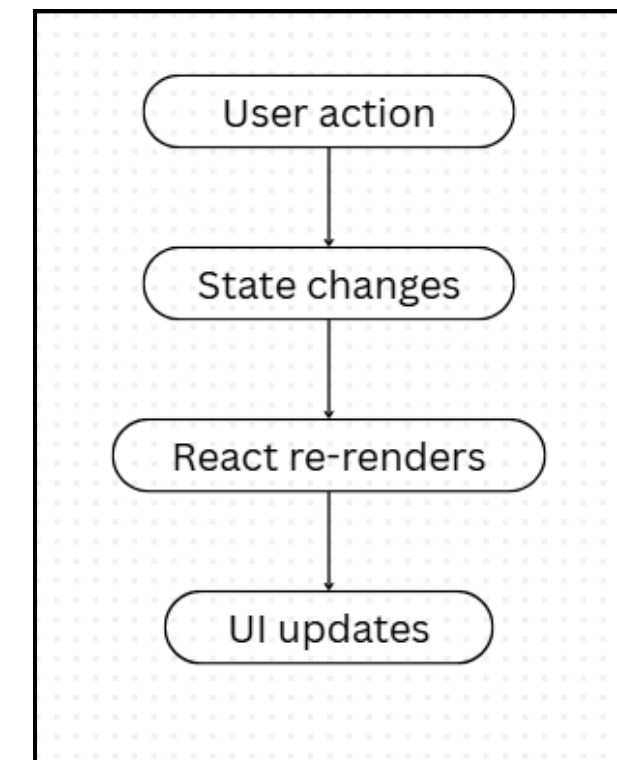
- **What is State?**

- Example:

```
const [count, setCount] = useState(0);  
const [email, setEmail] = useState("");
```

- **count** starts from 0
- **email** starts as empty string

How State Works:



State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **What is State?**
 - Example: Result

The screenshot displays a web application interface and its corresponding browser developer console. The application, titled "useState Result Example", shows a "Count value: -1" with "Increase" and "Decrease" buttons. Below these is an input field containing "test@gmail.com" and a label "Email value: test@gmail.com". The developer console on the right shows a series of log messages for the count value, ranging from -1 to 2, and the email value, which is consistently "test@gmail.com". A warning message is also visible in the console: "A form field element should have an id or name attribute".

Log Message	Source
Email value: test@gmail.com	App.jsx:46
Count value: 2	App.jsx:42
Count value: 1	App.jsx:42
Count value: 2	App.jsx:42
Count value: 1	App.jsx:42
Count value: 0	App.jsx:42
Count value: -1	App.jsx:42

Warning: A form field element should have an id or name attribute

- When clicked → count increases → UI updates

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **What is State?**

- Example: Code — [src/components/UseStateExample.jsx](#)

1

```
import { useState, useEffect } from "react";

function UseStateExample() {
  const [count, setCount] = useState(0);
  const [email, setEmail] = useState("");

  const increaseCount = () => setCount((prev) => prev + 1);
  const decreaseCount = () => setCount((prev) => prev - 1);
  const handleEmailChange = (e) => setEmail(e.target.value);

  useEffect(() => {
    console.log("Count value:", count);
  }, [count]);

  useEffect(() => {
    console.log("Email value:", email);
  }, [email]);
}
```

2

```
return (
  <div className="flex flex-col min-h-screen bg-gray-50">
    <main className="flex-1 flex items-center justify-center px-4">
      <div className="w-full max-w-md bg-white rounded-2xl border p-6 space-y-6">
        <h2 className="text-2xl font-bold text-center text-gray-800">
          useState Result Example
        </h2>

        <div className="space-y-3 text-center">
          <p className="text-lg font-medium">
            Count value:
            <span className="ml-2 text-blue-600 font-bold">{count}</span>
          </p>

          <div className="flex justify-center gap-3">
            <button
              onClick={increaseCount}
              className="px-4 py-2 bg-blue-600 text-white rounded-lg hover:bg-blue-700 transition"
            >
              Increase
            </button>

            <button
              onClick={decreaseCount}
              className="px-4 py-2 bg-red-600 text-white rounded-lg hover:bg-red-700 transition"
            >
              Decrease
            </button>
          </div>
        </div>
      </div>
    </main>
  </div>
);
```

3

```
    <div className="space-y-3">
      <input
        type="email"
        placeholder="Enter email"
        value={email}
        onChange={handleEmailChange}
        className="w-full px-4 py-2 border border-gray-300 rounded-lg focus:outline-none focus:ring-2 focus:ring-blue-500"
      />

      <p className="text-gray-700 text-sm">
        Email value:
        <span className="ml-2 font-semibold text-green-600">
          {email || "-"}
        </span>
      </p>
    </div>
  </main>
</div>
);

export default UseStateExample;
```

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **When to Use State?**
 - Use state when data changes over time
 - Common Use Cases:
 - Form inputs (email, password)
 - Loading / error messages
 - UI toggle (show / hide)
 - Counters
 - Dynamic data

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **What are Props?**
 - Props (short for properties) : Data from Parent to Child
 - Data passed from a **parent component** → **child component**
 - Passed from parent to child
 - Read-only (cannot be changed in child)
 - Used to configure components
 - Makes components reusable

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **What are Props?**

- Example: Child Component

```
// Child receives and uses props
function UserCard({ name, role }) {
  return (
    <p>
      {name} is a <strong>{role}</strong>
    </p>
  );
}
```

- Example (Reusable Component):



Parent Component

```
{/* Parent passes props to child */}
<UserCard name="Dana" role="Student" />
```

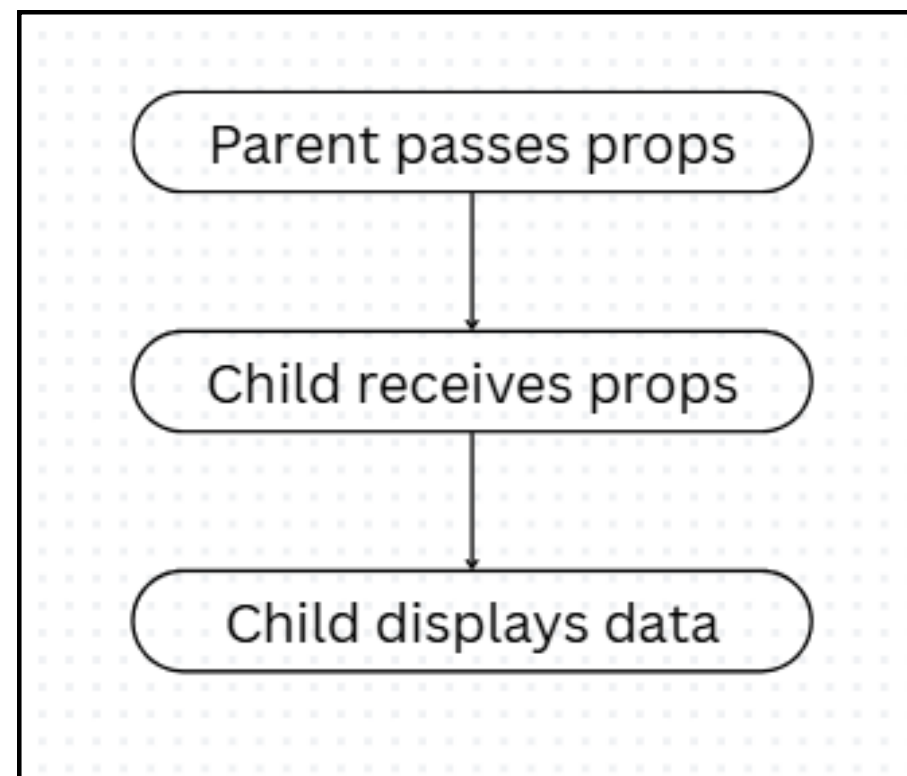
```
<UserCard name="Dana" role="Student" />
<UserCard name="Nita" role="Teacher" />
```

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **How Props Work**



- **When to Use Props?**

- Use props when you want to:
 - Pass data to child components
 - Configure components (customize UI)
 - Display dynamic data (user, product, etc.)
 - Reuse components with different data
 - Send functions (callbacks) to child

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **How Props Work**
 - **Important Rule:** Props are **READ-ONLY**

```
props.name = "New Name";
```

Use props only to display data

✗ WRONG

- **Props** are like arguments in a function

```
<UserCard name="Dara" />
```

like calling a function with parameter

State vs Props

React Data Management

React manages data in two main ways: **State** and **Props**

- **Props vs State**

Feature	Props	State
Source	Parent	Inside component
Changeable	 No	 Yes
Purpose	Pass data	Manage data

useState (State Hook)

What is useState?

- `useState` is a **React Hook** that allows a function component to **Store and manage** its own state (data)

Why useState?

- store data inside component
- update data over time
- automatically update UI

Import useState:

```
import { useState } from "react";
```

Basic Syntax:

```
const [value, setValue] = useState(initialValue);
```

- `value` → current state
- `setValue` → function to update state
- `initialValue` → starting value

useState (State Hook)

What is useState?

- Example:

```
const [count, setCount] = useState(0);
```

- `count` starts at 0
- `setCount()` changes the value
- **Important Rule:**
 - Always use `set`Value to update state

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <p>Count: {count}</p>  
  
      <button onClick={() => setCount((prev) => prev + 1)}>  
        Increase  
      </button>  
    </div>  
  );  
}
```

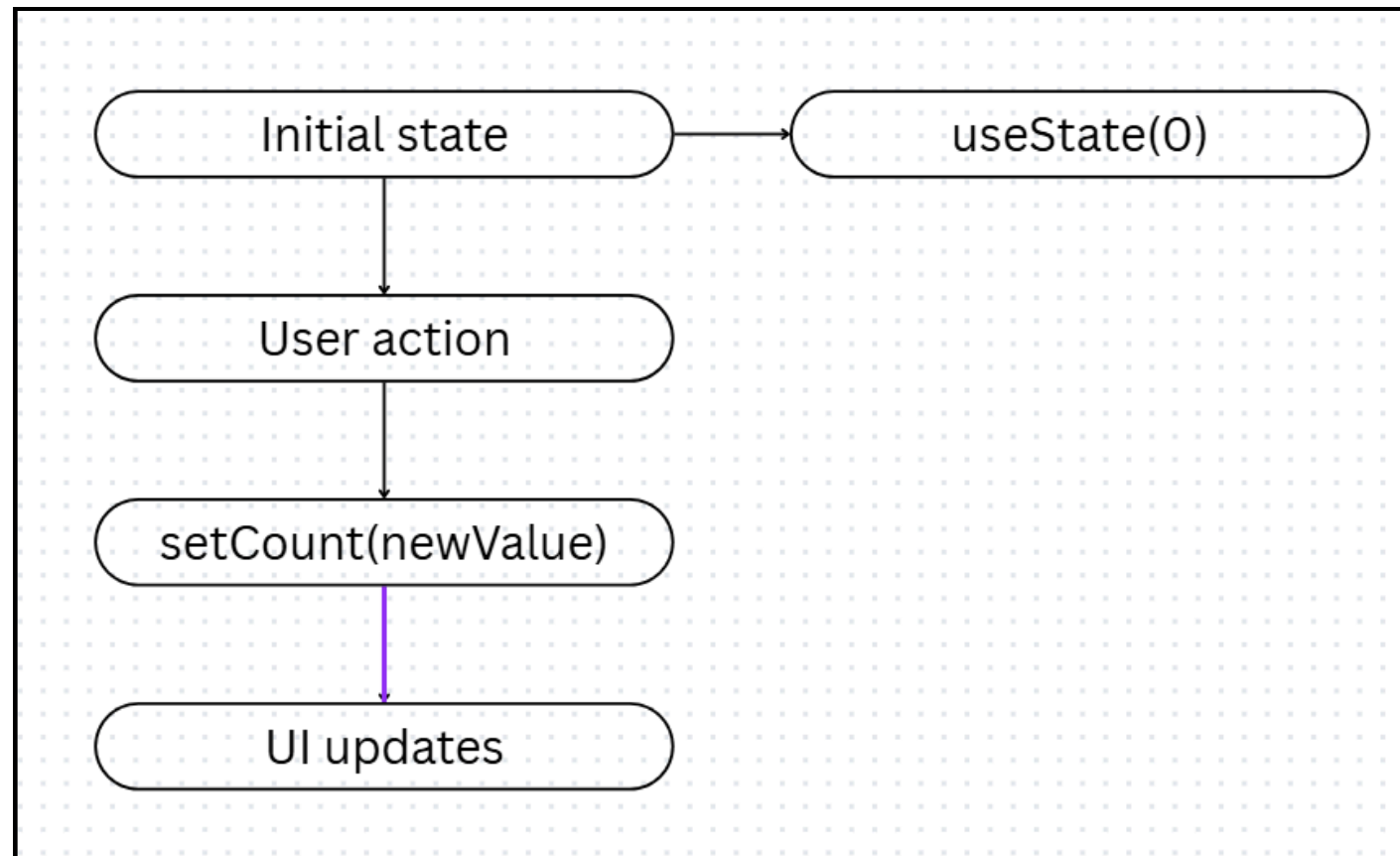
useState (State Hook)

What is useState?

- Example:

```
const [count, setCount] = useState(0);
```

- How It Works



How State Updates (useState Flow)

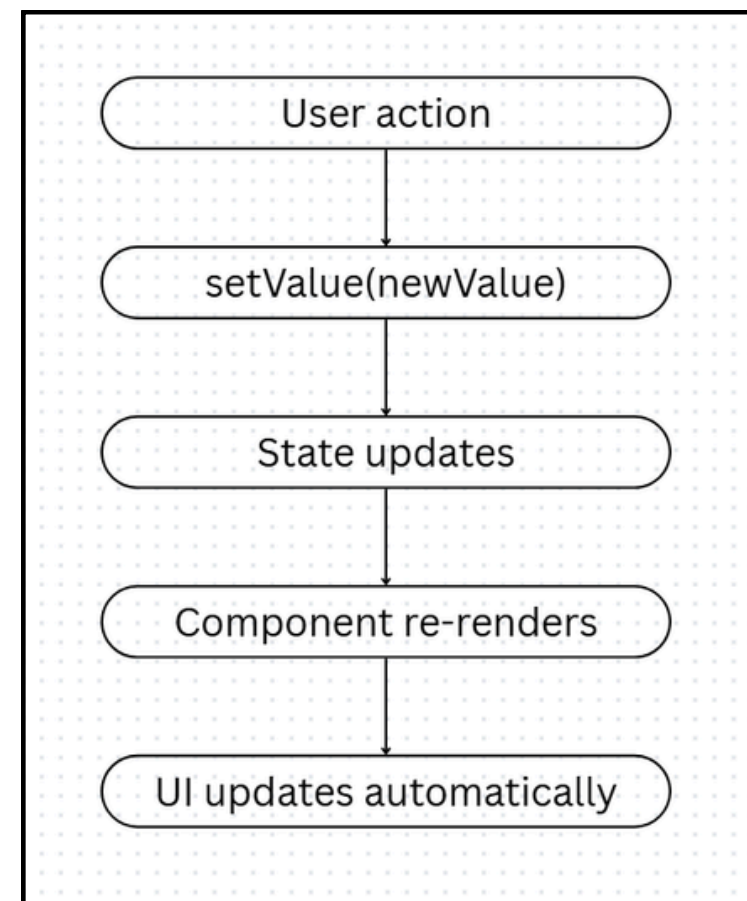
What happens when `set`Value() is called?

- When call:

```
set
```

Value(newValue);

- React will:
 - Update the state
 - Re-render the component
 - Update the UI with new value



How State Updates (useState Flow)

What happens when `setEmail()` is called?

- Example (Input Field)

```
const [email, setEmail] = useState("");  
  
<input  
  type="email"  
  value={email}  
  onChange={(e) => setEmail(e.target.value)}  
  placeholder="Enter email"  
>
```

1. **Initial State:** email starts as empty string ""

```
const [email, setEmail] = useState("");
```

2. **value={email}:** Input displays the value from state

3. **User Types:** `onChange` event is triggered

4. **State Updates:** State now stores new input value

```
setEmail(e.target.value)
```

5. **Component Re-renders:** React re-renders the component

6. **UI Updates:** Input shows updated value

- This is called a Controlled Component
 - Input value is controlled by React state

State Data Types

Common Data Types in useState

React state can store different types of data.

Data Type	useState Example	Common Use Case
Number	useState(0)	Counters, scores
String	useState("")	Form inputs, text
Boolean	useState(false)	Loading, toggles
Array	useState([])	Lists, todos
Object	useState({})	User data, settings

State Data Types

Common Data Types in useState

React state can store different types of data.

Example:

```
// Counter
const [count, setCount] = useState(0);

// Text input
const [email, setEmail] = useState("");

// Boolean toggle
const [isLoading, setIsLoading] = useState(false);

// Array of items
const [tasks, setTasks] = useState([]);
```

```
// Object (user profile)
const [user, setUser] = useState({
  name: "",
  email: "",
  age: 0,
});
```

Props

What are Props?

Props (short for **Properties**) are a way to pass data from a **parent component** → **child component**

Props are like **function parameters**

- **JavaScript Example:** `name` is a parameter

```
function greet(name) {  
  return "Hello " + name;  
}  
  
greet("John");
```

- **React Example (Component)**

```
function Greeting(props) {  
  return <h1>Hello {props.name}</h1>;  
}
```

```
<Greeting name="John" />
```

`name` is a prop

Props

Props are Read-Only

- Props **cannot be modified** inside a component

```
props.name = "New Name";
```

✗ WRONG

One-Way Data Flow: Parent → Child — Data only flows **downward**

Props

Props vs Function Arguments

- JavaScript Function Example — **a** and **b** are parameters

```
function add(a, b) {  
  return a + b;  
}  
  
add(5, 3);
```

- React Component Example

```
function Card(props) {  
  return <h1>{props.title}</h1>;  
}
```

Using the Component — **title** is a prop

```
<Card title="My Title" />
```

Props

How Props are Received

- Props are received as an object

```
props = {  
  title: "My Title"  
}
```

- **Method 1:** Without Destructuring (Longer)

```
function Card(props) {  
  return <h1>{props.title}</h1>;  
}
```

- **Method 2:** With Destructuring (Shorter)

```
function Card({ title }) {  
  return <h1>{title}</h1>;  
}
```


Props Data Types

Props Can Store Different Data Types

- Props can pass any JavaScript data type from parent to child.

Data Type	Example	Description
String	<code><Card title="Hello" /></code>	Text data
Number	<code><Card count={5} /></code>	Numeric values
Boolean	<code><Card isActive={true} /></code>	True / False values
Object	<code><Card user={{ name: "John", age: 25 }} /></code>	Structured data
Array	<code><Card items={[1, 2, 3]} /></code>	Lists of data
Function	<code><Card onClick={handleClick} /></code>	Event handler / callback

Props Syntax

Passing Props (Parent Component)

- Props are passed from parent → child using attributes.
 - Example (Parent.jsx) — `name`, `age`, `city` are props

```
function Parent() {  
  return (  
    <Child name="John" age={25} city="New York" />  
  );  
}
```

Props Syntax

Passing Props (Parent Component)

- Props are passed from parent → child using attributes.

- Example Receiving Props:

- Method 1: Without Destructuring

```
function Child(props) {  
  return (  
    <div>  
      <p>Name: {props.name}</p>  
      <p>Age: {props.age}</p>  
      <p>City: {props.city}</p>  
    </div>  
  );  
}
```

Access props using `props.property`

- Method2: With Destructuring

```
function Child({ name, age, city }) {  
  return (  
    <div>  
      <p>Name: {name}</p>  
      <p>Age: {age}</p>  
      <p>City: {city}</p>  
    </div>  
  );  
}
```

Side Effect Hook (useEffect)

What is Side Effect?

- A **side effect** is: **Code that runs** outside of rendering UI
- **Examples of Side Effects**
 - Fetch data from API
 - Check user login/session
 - Save data to localStorage
 - Set timers (setTimeout, setInterval)
- **Why useEffect?**
 - run code after render
 - handle async tasks
 - sync UI with external data

useEffect Hook

What is useEffect?

- `useEffect` is a React Hook that runs code **after the component renders (after UI updates)**

Simple : Render UI → `useEffect` runs

Import `useEffect` :

```
import { useEffect } from "react";
```

Basic Syntax:

```
useEffect(() => {  
  // side effect code here  
});
```

Example:

```
useEffect(() => {  
  console.log("Component rendered");  
});
```


useEffect Hook

useEffect with Dependency

- Syntax

```
useEffect(() => {  
  // code  
}, [dependency]);
```

- Example

```
useEffect(() => {  
  console.log("Count changed:", count);  
}, [count]);
```

- Runs only when **count** changes

Dependency Types

Dependency	Behavior
No array	runs every render
[]	runs once (on mount)
[value]	runs when value changes

useEffect Hook

useEffect with Dependency

- Example (Run Once)

```
useEffect(() => {  
  console.log("Run once when component loads");  
}, []);
```

- Example (Fetch Data)

```
useEffect(() => {  
  fetch("https://api.example.com/data")  
    .then(res => res.json())  
    .then(data => console.log(data));  
}, []);
```

- Example (Timer)

```
useEffect(() => {  
  const timer = setInterval(() => {  
    console.log("Running...");  
  }, 1000);  
  
  return () => clearInterval(timer);  
}, []);
```

- Example check session

```
useEffect(() => {  
  async function checkSession() {  
    const { data } = await api.getSession();  
    setUser(data?.session?.user || null);  
    setLoading(false);  
  }  
  
  checkSession();  
}, []);
```



THANK YOU

For your attention !

